

Week 3: Real GitHub OAuth Evidence

Saksham Boran

Roll No: 1024030856

12 September 2026

1 Context

Week 2's profile page displayed whatever GitHub URL a user had typed in during onboarding – an unverified string, not evidence. This week replaced that placeholder with a real GitHub OAuth connection, reused across the dashboard, onboarding's GitHub step, and the profile page, whose result merges directly into the user's skill ratings. The work spanned a hand-rolled OAuth flow, a database transaction, and a reconciliation rule shared with the existing resume-upload evidence path.

2 Backend: Keeping the Schema in Sync

Between this and the prior weeks, `Profile.skillRatings` and `Profile.activeResumeId` had been added to the schema by migrations landing alongside the resume-parsing work. Before starting on GitHub evidence, pulled those migrations and re-synced my local Neon database (`prisma migrate deploy / prisma generate`) so the Prisma client and the actual database agreed on the shape of `Profile` before writing any code against it.

3 Backend: Hand-Rolled GitHub OAuth

GitHub was never a sign-in provider – Week 2's Google-only, college-domain-restricted sign-in stayed as the only auth method. GitHub is purely an *evidence* source, connected after a user is already signed in, so it stays outside NextAuth entirely and is implemented as two plain route handlers.

3.1 Design: Two API Routes

- GET `/api/github/start` – generates a random CSRF `state` value, stores it in an `httpOnly` cookie alongside a `returnTo` value (also cookie-stored), and redirects to GitHub's OAuth authorize URL.
- GET `/api/github/callback` – GitHub redirects here with `?code&state`. The route validates `state` against the cookie, exchanges `code` for an access token server-side (the client secret never touches the browser), fetches the user's public repos, derives skills from the repos' primary languages, and writes the result to the database in one transaction.

3.2 Security Detail: The `returnTo` Allowlist

“Connect GitHub” appears in three places (dashboard, onboarding, profile), and after the OAuth round-trip the callback needs to send the user back to whichever one they started from. Rather than trusting an arbitrary `returnTo` query value – an open redirect if done naively – it's checked against a fixed allowlist before being used to build the final redirect:

```
const RETURN_TO_ALLOWLIST = ["/dashboard", "/profile", "/onboarding"];

export function sanitizeReturnTo(value, fallback) {
  if (value && RETURN_TO_ALLOWLIST.includes(value)) {
    return value;
  }
  return fallback;
}
```

3.3 Deriving Skills from Public Repos

`fetchPublicRepoLanguages` calls the GitHub repos API for a connected user’s own (non-fork) repositories and reads each repo’s single primary `language` field – a full byte-count breakdown per repo was considered and rejected, since it would need one extra API call per repository for marginal accuracy. Forks are excluded so a starred/forked repo doesn’t misattribute someone else’s work to the connecting user.

`deriveSkillsFromLanguages` then ranks languages by frequency and maps GitHub’s linguist language names onto the platform’s fixed skill-slug list (`GITHUB_LANGUAGE_TO_SKILL` in `lib/skills.ts`). Languages with no corresponding skill slug (e.g. “HTML”, “Shell”) are deliberately dropped rather than guessed at, and only true linguist-detected languages are mapped here – frameworks and tools like React or Docker aren’t detectable this way and are left to the resume-parsing path instead.

4 Backend: Reconciling Skill Ratings Across Two Evidence Sources

The core design decision this week: skill ratings should be *reconciled*, not overwritten, regardless of which evidence source produces them. `mergeSkillRatings` (moved from `lib/resume-upload.ts` into the shared `lib/skills.ts`, since it’s now used by both the resume and GitHub paths) only fills in skills the user doesn’t already have a rating for:

```
export function mergeSkillRatings(existing, newSkills) {
  const merged = { ...existing };
  for (const slug of newSkills) {
    if (!merged[slug]) {
      merged[slug] = "INTERMEDIATE";
    }
  }
  return merged;
}
```

This means a resume-derived rating and a GitHub-derived rating for the same skill never fight each other, and a user’s own self-reported rating (if they already had one from Path B onboarding) is never silently downgraded by a new evidence source.

4.1 Transaction: One Evidence Record per Account

Unlike resumes, where a user can have several historical uploads, a GitHub connection is treated as singular – reconnecting replaces rather than accumulates. The callback route wraps the delete-then-create-then-upsert sequence in a single `prisma.$transaction`:

```
await tx.evidenceRecord.deleteMany({ where: { userId, source: "GITHUB" } });
const created = await tx.evidenceRecord.create({
  data: { userId, source: "GITHUB", payload },
});
```

```
});
await tx.profile.upsert({
  where: { userId },
  create: { userId, githubUsername, skillRatings: mergedSkillRatings },
  update: { githubUsername, skillRatings: mergedSkillRatings },
});
```

This also cleanly clears out the old unverified-URL placeholder record the first time a user connects for real, and the transaction guarantees the evidence record and the merged skill ratings never end up out of sync if one write fails partway through.

5 Frontend Consequence: Surviving a Full-Page Redirect

Because OAuth requires a full-page navigation away to GitHub and back, any in-progress client state is lost by default. Onboarding’s multi-step wizard needed its state persisted to `sessionStorage` before redirecting, so a user connecting GitHub mid-onboarding returns to the same step with their other answers intact rather than starting over. Also built a shared `ConnectGithubButton` used identically across the dashboard, onboarding’s GitHub step, and the profile page, rather than three slightly different implementations of the same start-the-OAuth click handler.

6 Verification

No automated test framework exists yet, so verification was manual: TypeScript compiles cleanly, the production build succeeds, and a live walkthrough against the real Neon database and a real GitHub OAuth App confirmed:

- Connecting GitHub from the profile page, dashboard, and onboarding’s GitHub step all round-trip correctly and land back on the originating page.
- A connection creates exactly one `GITHUB EvidenceRecord` and merges derived language skills into `Profile.skillRatings` without touching any rating the user or a resume upload had already set.
- Reconnecting a different GitHub account replaces the old evidence record rather than leaving two.

7 Outcome

Shipped the full GitHub OAuth evidence connection – covering `lib/github.ts`, both API routes, the shared `ConnectGithubButton`, and the skill-ratings reconciliation now shared with the resume path – replacing the onboarding-only unverified-URL placeholder everywhere it appeared.

8 Follow-ups

- GitHub-derived skills default to `INTERMEDIATE` like resume-derived ones; there’s no signal yet (e.g. repo count or recency) that would justify a different starting level.
- The public-repos scan is capped at 100 repos per user (`MAX_REPOS_SCANNED`); untested against an account with more than that.

- Access tokens are used once during the callback and discarded – nothing is stored long-term, so re-deriving skills later (e.g. on a schedule) would require the user to reconnect.